



Technical Documentation

Unified Build Orchestration Script

Team Oriented Project

Group A

Linux Kernel Extension for DM-Verity Authentication, Verification and Setup

Date: **NOVEMBER 2025**

Cooperation with:

**BMW
GROUP**



ROLLS-ROYCE
MOTOR CARS LTD

[BMW Car IT, Ulm](#)

Eva-Helena Zorman

Tobias Kaufmann

Philipp Graf

1. Introduction

1.1. Purpose

This document provides technical specifications and operational details for the master orchestration script, **build_artifacts.sh**. This script unifies and manages the execution sequence of the project's core build processes, ensuring that all security-critical artifacts—specifically the verified root filesystem image—are created successfully and in the correct order.

1.2. Background and Project Context

In the project's secure boot environment, the generation of the final verifiable root filesystem image requires two distinct, sequential steps: building the base filesystem data, and then generating the dm-verity integrity metadata. This script acts as the single point of entry, enforcing the necessary dependency chain and providing clear pass/fail status feedback.

1.3. Access

The script discussed in this document, **build_artifacts.sh**, is designed to be executed from its location in the project's working directory.

- **Dependency Scripts:** It orchestrates two other scripts located in the same directory: `build_rootfs.sh` and `generate_verity.sh`.

2. Script Specifications: build_artifacts.sh

2.1. Overview

`build_artifacts.sh` is a pipeline orchestration script written in Bash. It uses the `set -euo pipefail` command to ensure strict error handling, guaranteeing that any failure in a sub-process immediately halts the entire build. The script's output uses ANSI color codes (GREEN, RED, YELLOW) to provide clear, actionable feedback on the status of each stage.

2.2. Build Stages and Dependency Management

The script executes a three-stage sequential workflow:

Stage [1/3]: Building Root Filesystem

This stage executes the base filesystem creation script (`build_rootfs.sh`), which is responsible for installing Debian, configuring user accounts, calculating the disk size with reserved dm-verity slack space, and creating the initial ext4 image.

The script ensures that Stage 1 must complete successfully (exit 0) before the process can continue to the next stage:

```
32 # [1] Build rootfs
33 echo -e "${YELLOW}[1/3] Building Root Filesystem...${RESET}"
34 if bash "$BUILD_DIR/build_rootfs.sh"; then
35     echo -e "${GREEN}✓ Root filesystem built successfully${RESET}"
36 else
37     echo -e "${RED}✗ Failed to build rootfs${RESET}"
38     exit 1
39 fi
```

Stage [2/3]: Generating dm-verity Metadata

This stage executes the integrity generation script (generate_verity.sh). This script depends entirely on a successfully created and correctly sized rootfs.img from Stage 1. It performs the complex operations of generating the hash tree and embedding the dm-verity metadata, completing the image integrity preparation.

A failure in this stage immediately terminates the master script:

```
41 # [2] Generate dm-verity metadata
42 echo -e "${YELLOW}[2/3] Generating dm-verity metadata...${RESET}"
43 if bash "$BUILD_DIR/generate_verity.sh"; then
44     echo -e "${GREEN}✓ dm-verity metadata generated successfully${RESET}"
45 else
46     echo -e "${RED}✗ Failed to generate dm-verity metadata${RESET}"
47     exit 1
48 fi
```

Stage [3/3]: Verification and Output

After both critical build stages complete, the script performs a final verification check to ensure the core artifact, \$ROOTFS_IMG, exists and provides the final output path to the user:

```
50 # Verify rootfs image exists
51 if [[ ! -f "$ROOTFS_IMG" ]]; then
52     echo -e "${RED}✗ rootfs.img not found at $ROOTFS_IMG${RESET}"
53     exit 1
54 fi
55
56 echo -e "${YELLOW}rootfs.img ready at:${RESET} $ROOTFS_IMG"
```

3. Technical Benefits

3.1. Enforced Sequential Dependencies

By executing the dependency scripts within if blocks that enforce an immediate exit 1 upon failure, the master script guarantees that the dm-verity process never runs on an incomplete or

corrupted base filesystem. This is crucial for maintaining the integrity chain and build reliability.

3.2. User Experience and Reporting

The use of ANSI escape sequences (colors) provides a clear, high-contrast visual report to the user, instantly identifying which of the three steps failed or succeeded (e.g., green for success, red for failure). This enhances the professionalism of the build process and facilitates rapid troubleshooting.